

Learned Page Replacement in xv6

Can a small learned model choose which page to evict better than Clock?

Summary

What we did. When memory is full, the operating system must choose a page to move to disk. A bad choice evicts a page that is needed again soon and causes another page fault. xv6, like most kernels, uses fixed rules for this; its best one is *Clock*. We trained models to make this choice from recorded memory traces, using only signals a real kernel can see, and ran the best model inside the xv6 kernel as a new eviction policy, `VM_POLICY_ML`.

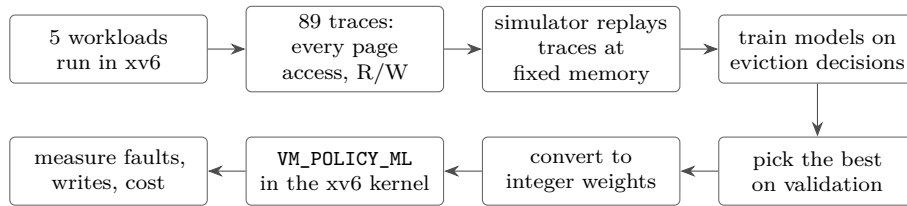


Figure 1: The whole pipeline. Sections 1–3 describe the steps; Section 4 the results.

How we did it (Figure 1):

- Recorded every page access of five benchmark programs in xv6 (a B-tree, a key-value store, graph algorithms, a sort, a matrix multiply): 89 traces, 123 million accesses.
- Replayed the traces in a simulator at 5, 10 and 20% of each program’s memory, and trained models to predict how soon each page will be used again.
- Tried 300 combinations of 12 page features (every combination within each feature group) and five model types (linear, MLP, ranking MLP, GRU, embedding); chose the best on validation data only, then tested on new runs and on new program variants.
- Converted the chosen linear model to integers (xv6 has no floating point) and measured it in the kernel: 231 runs, 7 policies.

What we found.

- The kernel’s own signals are enough. Knowing every memory access helps almost nothing; the most useful signal is how long a page has gone unused.
- A simple linear model (a weighted sum of 4–5 signals) is as good as the neural models, and costs only a few multiply-adds per page.
- Integer arithmetic loses nothing compared to floats.
- The simulator predicts the kernel’s results closely.

Final result. Inside xv6 the learned linear policy causes fewer page faults than Clock on every benchmark, on runs and variants it never saw, and up to 96.5% fewer disk writes (Table 1).

What it costs, compared with Clock.

- *Slower decisions.* The policy scores every page in memory at every eviction: about 43 timer ticks plus 3.9 per page (1 tick = 0.1 μ s). Choosing a victim takes 8.8 μ s with 10 pages in memory and 101.2 μ s with 249 pages: 2.4–32.4 times Clock, which usually checks only a few pages, and 1.8–2.5 times xv6’s Aging policy.
- *More memory.* 33 bytes of bookkeeping per physical page (about 1 MB for xv6’s 32,768 pages, under 1% of its 128 MB) and 12 bytes per swap slot (96 KB).

- *One model does not fit all.* A single global model loses to Clock on matmul (11.5% more faults); the best results need a model per program.
- *Not yet measured:* whether the fewer faults and saved disk reads outweigh the slower decisions in total program run time.

Benchmark	Page faults vs Clock	Disk writes vs Clock	Decision time per eviction
	test / held-out	test / held-out	ML vs Clock
btree	−17.7% / −14.6%	+2.2% / −3.4%	101.2 μs vs 3.2 μs
graph	−38.6% / −45.9%	−95.5% / −86.2%	59.4 μs vs 5.6 μs
kv	−12.9% / −12.5%	−31.8% / −12.5%	27.5 μs vs 2.8 μs
matmul	− / −2.7%	− / −96.5%	9.2 μs vs 3.8 μs
sort	−4.0% / −6.7%	−5.6% / −7.5%	8.8 μs vs 3.5 μs

Table 1: Results at a glance: the per-workload **linear** model running inside xv6 at 10% memory, compared with Clock on the same runs. Test: new runs of a known program; held-out: program variants never seen in training (matmul has no test traces: its seed does not change which pages it touches, so each variant was recorded once; $n = 64$ and 96 are used for training and $n = 128$ is held out). Decision time: test traces (matmul: held-out), 10 MHz kernel timer.

1 Traces and benchmarks

1.1 The benchmarks

We use five programs that run natively in xv6 (`user/*bench.c`); a sixth, `lzwbench`, exists but is not used (Table 2). Real applications such as Redis or SQLite cannot run on xv6 (no network, no floating point, small files), so each benchmark reproduces the *memory-access pattern* of a real kind of program. Each one was built so that no single simple rule (“evict the oldest”, “evict the least used”) is best for it; otherwise there would be nothing to learn.

Benchmark	What it does	Why it is hard for one fixed rule
btreebench	A B+tree with one node per 4KB page, like a database’s page store: inserts, lookups, range scans, a mix, optionally with a write-ahead log.	Lookups reuse the same top nodes (frequency matters), inserts touch new leaves once (recency matters), scans walk leaves in order.
kvbench	A hash table with popular and unpopular keys (Zipf), like a key-value store; read/write mixes A, B, C, D, F as in the YCSB benchmark.	The set of popular keys changes over time, so formerly hot pages must be let go.
graphbench	Breadth-first search and PageRank over a synthetic graph with a few very connected “hub” vertices.	BFS order depends on the data; PageRank revisits the hubs every iteration.
sortbench	Merge sort over an array larger than memory.	Mostly sequential; little room to beat simple rules. Used as a sanity check.
matmulbench	Integer matrix multiplication, naive and blocked.	Tiny working sets; the two orders have very different locality.
lzwbench*	LZW compression of a real English text (the GPLv3 licence).	Dictionary lookups follow the input text: some entries are reused constantly, most are used once.

Table 2: The benchmarks. *`lzwbench` is implemented but **not used** in this study (no training, validation, test or kernel runs): its trace (about 109 MB) is larger than the largest file xv6 can store (about 67 MB), so it cannot be recorded completely.

Each benchmark has several *variants* (different parameters, for example key-value mode A vs. F, or B-tree lookups vs. scans) and is run with several random *seeds*. In total there are 25 variants and 89 traces (Table A1 in the appendix lists them).

1.2 How a trace is recorded

Every benchmark reads and writes its data through one small accessor function. When tracing is on, that function also records one line per access: **R** <page> for a read or **W** <page> for a write. The result is the

complete sequence of page accesses, not only the page faults. This is necessary because the best possible eviction choice (Belady’s algorithm, below) needs to know when each page is used next, including uses that hit in memory.

The trace is written to a file inside xv6 and copied to the host afterwards. A trace does not depend on how much memory the program is given, so each trace is recorded once with plenty of memory, and the memory size is chosen later in simulation.

Each trace was checked before use: the run had to pass, the file had to have exactly the number of lines and bytes the program reported, a repeated run of the same seed had to produce a byte-identical trace (22 repeats, all identical), and replaying the trace under FIFO had to reproduce the kernel’s own eviction count.

1.3 The dataset

Each trace is stored with, for every access, the page, whether it was a write, and the distance to the next access of the same page (the label used for training, Section 2). The 89 traces hold 123 million accesses.

2 Models and features

2.1 What a model does

At every eviction, the policy looks at all pages currently in memory (the *candidates*), gives each a score, and evicts the page with the highest score. A model is trained to predict, for each candidate, how far in the future that page will be used next (as $\log(1 + \text{distance})$, in accesses). Evicting the page used furthest in the future is exactly what the optimal policy, Belady’s, does; the model tries to imitate it with the information it has.

2.2 Features: what the model is allowed to know

A kernel does not see individual memory accesses: the hardware handles them. It only sees page faults, and two bits per page that the hardware sets: the *accessed* bit (the page was used) and the *dirty* bit (it was written). The kernel can read and clear these bits whenever it scans memory, which it does at each eviction. These two bits are the kernel’s only raw signal about how a page is used, but by recording them at every scan, together with its own events (when it loaded, evicted and reloaded a page), the kernel can build richer features at run time: for example how many scans a page has gone unused, or how often it was found in use. We define features in three tiers (Table 3); the kernel computes all of tiers K and K+ itself while the program runs.

Tier	Feature	Meaning	Why it is included
F (oracle)	rec	accesses since the page’s last use	classic recency (what LRU uses)
	freq	number of uses so far	classic frequency (what LFU uses)
	sd	distinct pages used since its last use	stack distance
	wr	share of its uses that were writes	write-heavy pages cost a disk write
K (kernel)	ref	accessed bit at this scan	what Clock uses
	aging	8-bit history of accessed bits	what xv6’s Aging uses
	sfreq	scans that found it accessed	kernel version of frequency
	idle	scans since it was last found accessed	kernel version of recency
	age	scans since it was loaded	how new the page is
	dirty	dirty bit	evicting a dirty page costs a disk write
K+ (refault)	refaults	times it was evicted and came back	a page that keeps returning is needed
	rdist	evictions between its last eviction and its return	Linux already measures this (“refault distance”)

Table 3: The twelve features. Tier F needs every access, so no kernel can compute it; it is used only to measure how much a kernel loses by not seeing accesses. Tiers K and K+ are what a deployable policy can use. A *refault* is a page fault on a page that was evicted earlier. Counts enter as $\log(1 + \text{count})$.

2.3 Model types

Model	What it is	Why it was tried
Linear	a weighted sum of the chosen features	cheap enough for a kernel: a few multiply-adds per page
MLP	small neural network, 2 layers of 32	tests whether non-linear combinations help
Ranking loss	linear or MLP trained to put the highest score on Belady’s choice within each decision	trains directly on the decision rather than on distances
GRU	recurrent network over a page’s last 8 gaps between uses	tests whether a page’s access rhythm helps
Embedding	a learned vector per page plus the last 16 pages used by the program	tests whether knowing which pages are used together helps

Table 4: Model types, from cheapest to most expensive.

Each model is trained either *per workload* or *globally*. A per-workload model is trained on the training traces of one benchmark, across all its variants except the held-out one (the btree model learns from insert, lookup, mixed and mixedwal, and is also tested on the unseen scan variant). A global model is trained on all five benchmarks together. The global model is the realistic case for a kernel that does not know which program is running.

2.4 From a trained model to a kernel policy

xv6 has no floating point, so the chosen linear models are converted to integers (`tools/ml2/export_kernel.py`):

- Each feature is computed as a fixed-point number, the real value times 256. The $\log(1 + x)$ is read from a 256-entry table, with a second table for large values (`kernel/mlfeat.h`).
- Each trained weight is divided by its feature’s training standard deviation, multiplied by 256 and rounded to an integer. The mean and the constant term are the same for every candidate, so they do not affect which page has the highest score and are dropped.
- The score of a page is $\sum_j q_j X_j$: one integer multiply-add per feature.

Table 5 shows what the precision means for one weight, and Table 6 shows that 8 bits lose nothing: every selected model’s result stays within ± 0.001 of the float version.

Setting	Multiply by	Rounding step	0.4567 becomes	Stands for (error)
4-bit	16	$1/16 = 0.0625$	7	0.4375 (0.0192)
8-bit	256	$1/256 \approx 0.0039$	117	0.45703 (0.0003)
12-bit	4,096	$1/4096 \approx 0.00024$	1,871	0.45679 (0.00009)

Table 5: Integer weight precision. Since the xv6 kernel does not use floating point, each trained weight w (already divided by its feature’s training standard deviation) is stored as the integer $\text{round}(w \cdot 2^b)$, where b is the number of fractional bits. A larger b gives a finer rounding step and a closer match to the trained model. Features always use 8 fractional bits (they are scaled by 256). The kernel uses $b = 8$; the graph model’s weights, for example, include 159 and -347 . The table shows the rounding of an example weight, 0.4567, at each precision.

The policy, `VM_POLICY_ML`, is the function `choose_ml()` in `kernel/vmpage.c`. At each eviction it scans every candidate (reads and clears its accessed bit, updates its counters), scores it, and evicts the highest score. Two rules complete it:

- **Probation.** A page loaded fewer than p scans ago is not evicted while an older candidate exists (p is 0–4, chosen per model on validation). A brand-new page has no history yet, so its score is least reliable; this is the same idea as Clock’s second chance.
- **Ties** go to the page loaded earliest.

The default features need a little memory per swap slot: when a page is written to swap, the kernel stores the current scan count and the page’s default count with the slot, and reads them back when the page returns. Weights are per process: `vmctl(VM_SET_ML_WEIGHTS)` loads a model, and child processes inherit it.

Workload	Split	float	4-bit	8-bit	12-bit
btree	test	0.833	0.834	0.833	0.833
btree	held-out	0.863	0.875	0.862	0.863
graph	test	0.845	0.848	0.845	0.845
graph	held-out	0.798	0.812	0.799	0.798
kv	test	0.905	0.905	0.905	0.904
kv	held-out	0.905	0.903	0.904	0.904
matmul	held-out	0.681	0.681	0.681	0.681
sort	test	0.983	0.983	0.983	0.983
sort	held-out	0.933	0.932	0.933	0.933

Table 6: Float vs. integer scoring (faults relative to Clock, simulation, averaged over 5/10/20% memory). Compare each integer column with the float column on the same row: the closer, the less the rounding (Table 5) lost. Differences of about ± 0.001 are noise: rounding flips a few close eviction choices, sometimes for the better.

The default is the global model. The program `vmrun` runs any command under any policy and model, for example `vmrun ml kv kvbench ...`.

As an example, the selected graph model is $159 \cdot \text{idle} - 347 \cdot \text{dirty} - 2 \cdot \text{sfreq} + 24 \cdot \text{rdist}$ (probation 4 scans): evict pages that have been idle longest, and strongly prefer to keep dirty pages, which would cost a disk write. Figure 2 works through three pages.

	page A	page B	page C
kernel counters	idle 12, sfreq 3, clean	idle 1, sfreq 9, dirty, rdist 40	idle 6, sfreq 1, dirty
features ($\times 256$)	657, 355, 0, 0	177, 589, 256, 951	498, 177, 256, 0
score $\sum_j q_j X_j$	103,753 → evict	-39,043	-10,004

Figure 2: How the kernel scores pages, with the graph model’s integer weights (q : idle 159, dirty -347 , sfreq -2 , rdist 24). Features are listed as idle, sfreq, dirty, rdist; each is $\text{round}(256 \cdot \log(1 + \text{count}))$, or 0/256 for the dirty bit. Page A has gone 12 scans unused and is clean, so it gets the highest score and is evicted. Page C has also been idle, but it is dirty, and the large negative dirty weight keeps it in memory. Page B was used recently.

3 Training, validation and evaluation

3.1 Train / validation / test / held-out split

The split is fixed before any experiment, so every result is reported on the same data:

- **Held-out** (18 traces): five whole *variants* never seen in training (`kv-F`, `btree-scan`, `graph-bfs2000`, `sort-n60000`, `matmul-*128`). This tests a new kind of behaviour.
- In every other variant, the last seed is **test** (15 traces), the second-to-last is **validation** (15), and the rest are **train** (41). Test checks a new run of a familiar behaviour.

Models are trained on train, chosen on validation, and reported on test and held-out, which are never used for any choice. (matmul has one trace per variant, so its models are chosen on its training traces.)

3.2 Simulator

Training and most evaluation use a simulator (`tools/ml2/pagesim.c`), a program that runs on the host computer, not in xv6. It replays a trace with a fixed number of memory frames and counts page faults, evictions and disk writes. It implements every xv6 policy (FIFO, Clock, Aging, LFU), LRU, and Belady’s optimal policy. It was checked against independent code and against the kernel: FIFO, LRU and Belady agree exactly on all 534 trace $\times$ memory-size cases, and Clock, Aging and LFU match a direct copy of the kernel’s logic.

Memory sizes are 5%, 10% and 20% of the pages each trace touches: the smaller the memory, the higher the pressure, so 5% is the hardest case. The selected models were then also run inside xv6 itself (Section 3.5).

3.3 Training data

A model is used at eviction time, so it is trained on eviction decisions. Each training trace is replayed under LRU at the three memory sizes. At a random sample of evictions, up to 32 resident pages are recorded with all twelve features and their true next-use distance: always Belady’s choice, always the 4 most recently used pages (so the model sees pages that must be kept), and the rest at random. This gives 4.73 million rows from 357,000 decisions.

Linear models are fitted in closed form (weighted ridge regression). Neural models use Adam with early stopping on the validation loss.

3.4 Choosing models

For linear models, every combination of features was tried: 15 subsets of the oracle features, 255 subsets of the kernel features, and 30 mixed sets, each as a per-workload and a global model. That is 28,800 complete replays. For each tier, the five best subsets on validation were tried with probation 0, 1, 2 and 4, and the best combination on validation at all three memory sizes was kept. Only then were test and held-out replayed.

3.5 How performance is measured

- **Faults relative to Clock:** faults of the policy divided by Clock’s faults on the same trace and memory size. Below 1 is better than Clock. Clock is the baseline because it is the best policy xv6 has. Values are averaged (geometric mean) over traces.
- **Page writes relative to Clock:** the same for disk writes.
- **Inside the kernel:** every test and held-out trace was run in xv6 itself (QEMU, one CPU) at 10% memory under seven policies, 231 runs in total. These numbers are the kernel’s own counters. The kernel also counts the time spent choosing each victim (in ticks of its 10 MHz timer).

4 Results

4.1 How much room is there?

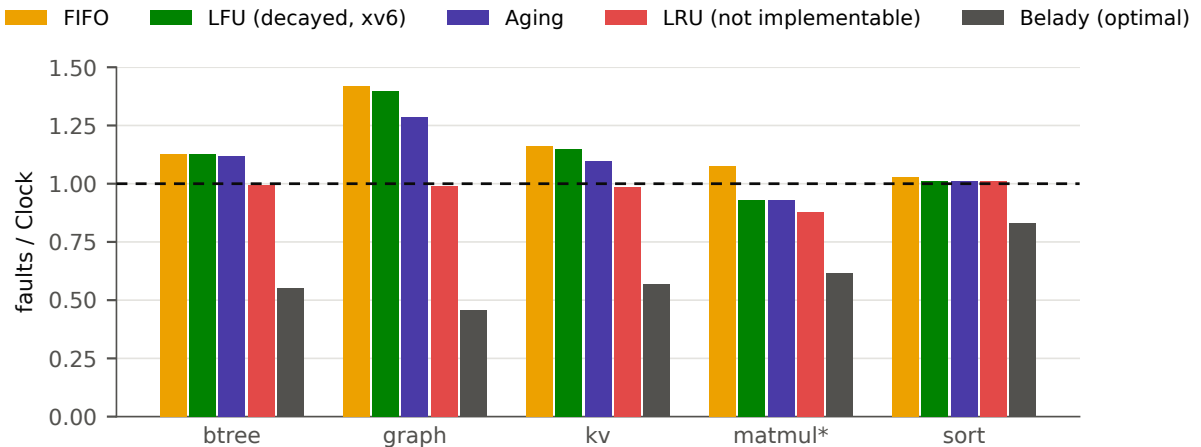


Figure 3: The policies a kernel has today. Each bar is a policy’s faults divided by Clock’s on the same traces (test traces, *matmul: held-out; averaged over 5/10/20% memory). The dashed line is Clock: above it is worse, below it is better. The dark bar is Belady, the best possible, so the gap between the line and the dark bar is the room for a better policy.

Clock is as good as true LRU on four of the five workloads (Figure 3). xv6’s Aging is only slightly better than FIFO, and its LFU is no better. The optimum needs 42.9%–54.4% fewer faults than Clock on btree, kv and graph, so there is a lot of room for a better policy there. On sort the room is small (17.0%).

4.2 Which signals matter

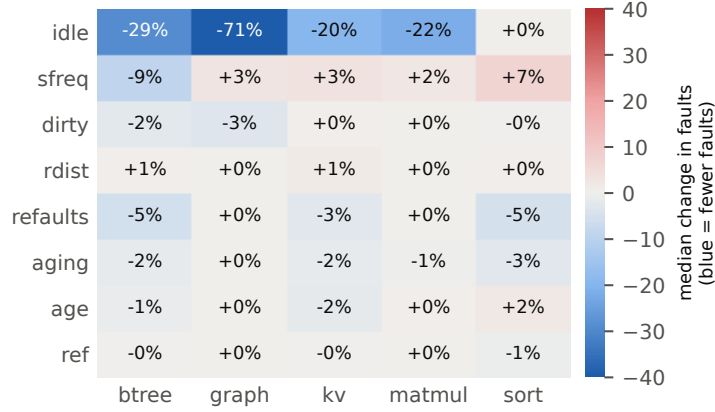


Figure 4: What each kernel feature adds: the median change in faults when the feature is added to a feature set that does not have it, over all such pairs of sets (per-workload models; test, matmul held-out).

`idle`, the number of scans since a page was last found in use, is by far the most useful kernel signal (Figure 4): adding it cuts faults by a median 70.8% on graph, 29.2% on btree, 19.5% on kv and 21.8% on matmul. It is the kernel’s version of recency, and it is also what Linux’s newest reclaim code (MGLRU) tracks. The other features add a little each, and which ones help depends on the workload.

The choice of features matters a great deal. Each of the 8 kernel features (K and K+) is either used or not, which gives $2^8 - 1 = 255$ non-empty feature sets. Of these, more than half are worse than Clock on kv and sort, and up to 126 are more than three times worse than Clock on graph. Selecting on validation is part of the method.

4.3 Kernel signals are enough

From here on, “the model” and “ML” mean the selected *linear* model; Section 4.4 compares the other model types.

Workload	per-workload model		one global model		optimum Belady (test)
	test	held-out	test	held-out	
btree	0.833	0.863	0.864	0.898	0.550
graph	0.845	0.798	0.874	0.792	0.456
kv	0.905	0.905	0.908	0.901	0.571
matmul	—	0.681	—	0.967	0.617 [†]
sort	0.983	0.933	1.005	0.966	0.830

Table 7: The selected **linear** model on kernel+refault (K∪K+) features, in simulation: faults relative to Clock, averaged over 5/10/20% memory. “Per-workload”: each benchmark’s own linear model; “one global model”: a single linear model trained on all benchmarks. [†]held-out.

Using only what a kernel can see, the selected model beats Clock on every workload, on new seeds and on new variants (Table 7), with fewer faults (test / held-out) on btree 16.7% / 13.7%, graph 15.5% / 20.2%, kv 9.5% / 9.5%, sort 1.7% / 6.7% and matmul 31.9% (held-out). It also makes far fewer disk writes. Evicting a modified (dirty) page means writing it to disk first; an unmodified page can simply be dropped. The model sees the dirty bit and prefers unmodified pages, so on graph it makes 58.2% fewer writes than Clock on test traces and 66.2% fewer on held-out.

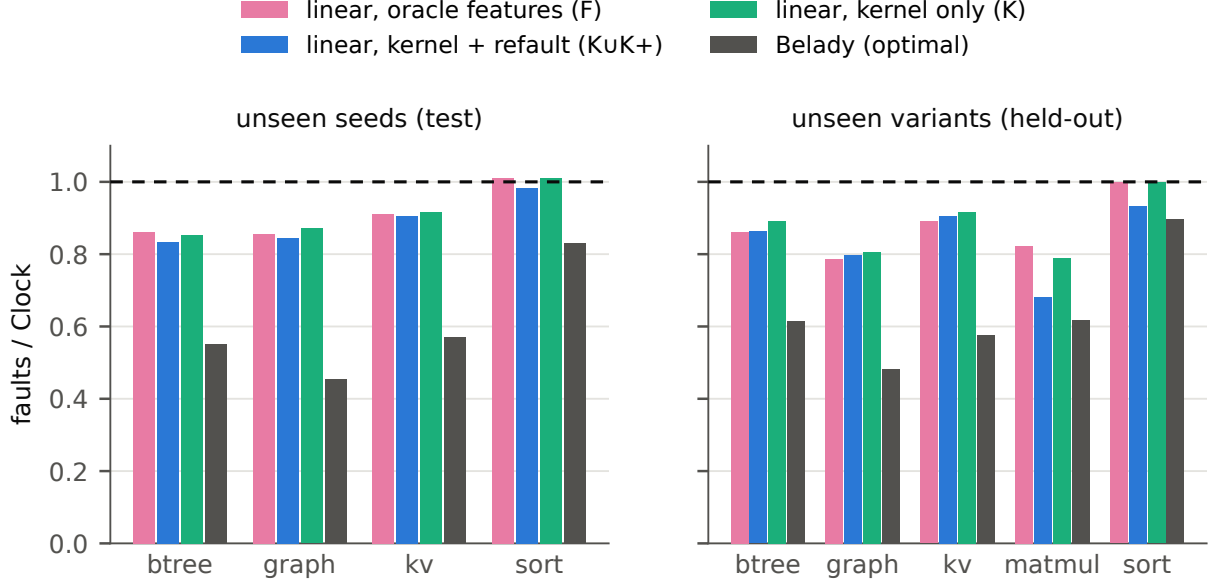


Figure 5: The best **linear** model of each feature tier, chosen on validation, relative to Clock (averaged over 5/10/20% memory). Lower is better.

Knowing every access does not help much (Figure 5): on btree, graph and kv the best oracle model is between 1.6% better and 3.3% worse than the kernel model, and it is worse on matmul and sort. The refault features are worth having: adding them improves the kernel-only model in all nine workload/split cases.

One global model keeps most of the gain on btree, graph and kv, but little on matmul (0.967 vs. 0.681 of Clock’s faults).

The gain depends on memory pressure. On graph (test) the model needs 0.624 of Clock’s faults at 10% memory, but 0.979 at 5% and 0.986 at 20%.

4.4 Bigger models do not pay

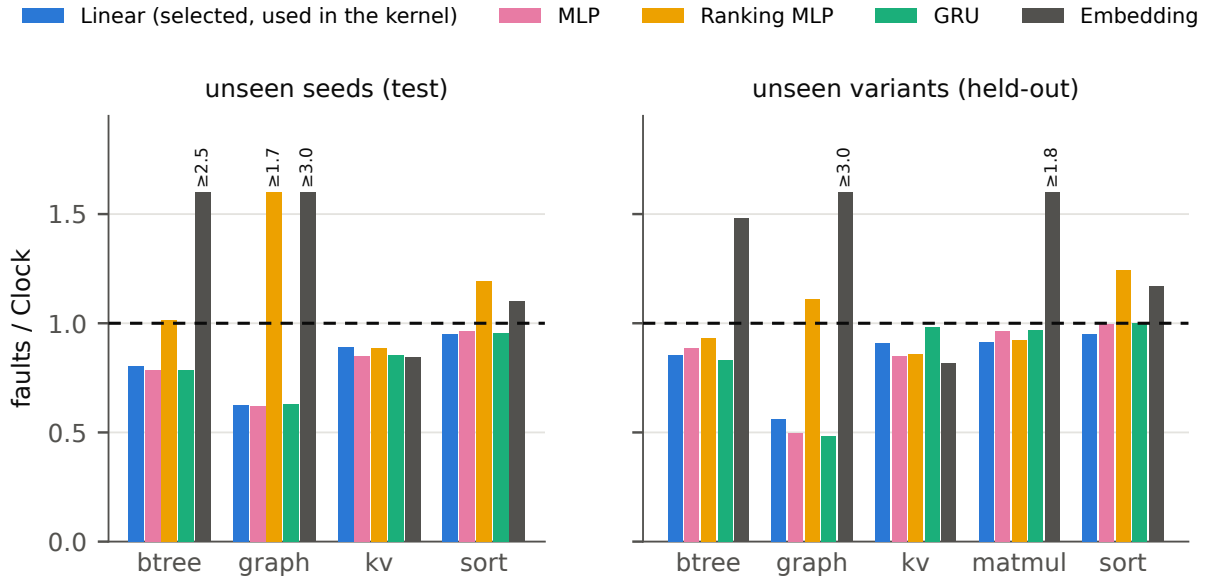


Figure 6: The five model types compared: faults relative to Clock (dashed line), each benchmark’s own model, 10% memory. Bars that would go off the chart are cut at 1.6 and labelled with their value ($\geq$: the run was stopped once it reached that many times Clock’s faults).

Workload	Linear (selected)	MLP	Ranking MLP	GRU	Embedding
btree	0.803	0.786	1.014	0.784	≥ 2.456
graph	0.624	0.622	≥ 1.699	0.632	≥ 3.000
kv	0.890	0.852	0.887	0.854	0.847
sort	0.950	0.965	1.192	0.957	1.103

Table 8: Model types compared: faults divided by Clock’s (below 1 is better), per-workload models, test traces, 10% memory. Each column is a different model type, each trained per workload. *Linear (selected)* is the linear model with the feature set and probabtion chosen on validation, the one used in Table 7 and in the kernel. MLP and ranking MLP use the KUK+ features; GRU and embedding use access history. $\geq$: the run was stopped once it reached $3\times$ Clock’s faults.

The neural models are not clearly better than the linear one (Figure 6, Table 8). An MLP is somewhat better on kv (0.852 vs. 0.890 of Clock’s faults), btree (0.786 vs. 0.803) and held-out graph (0.495 vs. 0.560), and worse on sort. Training on the ranking of each decision is worse. The embedding model, which looks at which pages the whole program used recently, is among the best on kv but fails badly on btree and graph. A linear score is therefore the right choice for a kernel: it is as good and costs a few multiply-adds per page.

Integers lose nothing. With 8-bit integer weights and fixed-point features, every selected model’s result is within ± 0.001 of the float version; with 4-bit weights, within 1.7% (Table 6, Section 2.4).

4.5 Inside the xv6 kernel

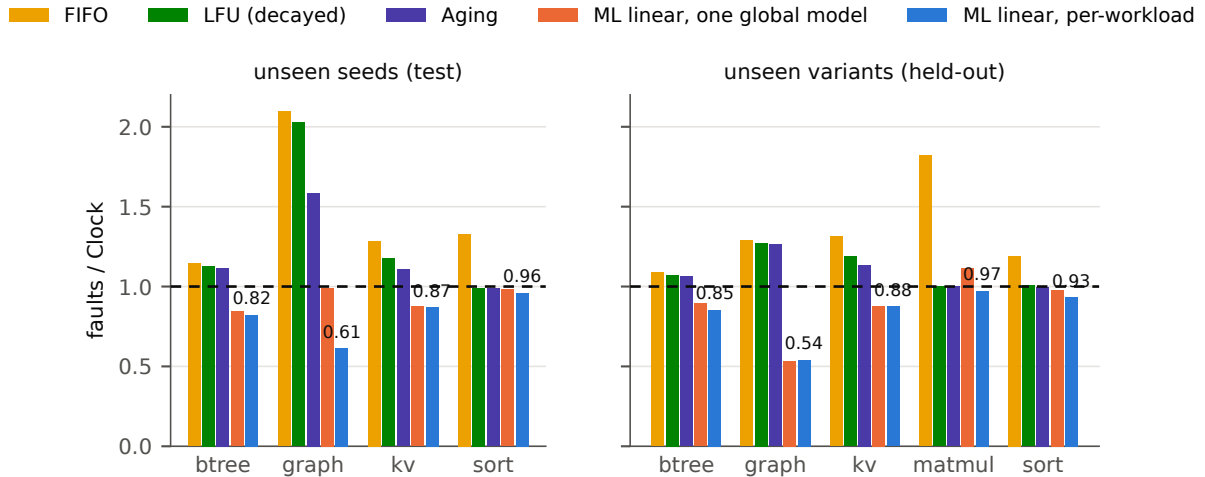


Figure 7: Measured in xv6: page faults relative to Clock (dashed line) at 10% memory, kernel counters, 231 runs. “ML” is the selected **linear** model with integer weights (the only model type run in the kernel): blue is each benchmark’s own model, orange the single global model. Numbers: the blue bars.

Workload	faults / Clock		writes / Clock		global model, faults	
	test	held-out	test	held-out	test	held-out
btree	0.823	0.854	1.022	0.966	0.843	0.893
graph	0.614	0.541	0.045	0.138	0.989	0.535
kv	0.871	0.875	0.682	0.875	0.875	0.878
matmul	—	0.973	—	0.035	—	1.115
sort	0.960	0.933	0.944	0.925	0.980	0.977

Table 9: VM_POLICY_ML (the selected **linear** models, integer weights) measured in xv6 at 10% memory, relative to Clock. First four columns: each benchmark’s own model; last two: the single global model.

Inside the kernel the learned policy beats Clock on every workload, on new seeds and new variants (Figure 7, Table 9): fewer faults (test / held-out) on btree 17.7% / 14.6%, graph 38.6% / 45.9%, kv 12.9% / 12.5%,

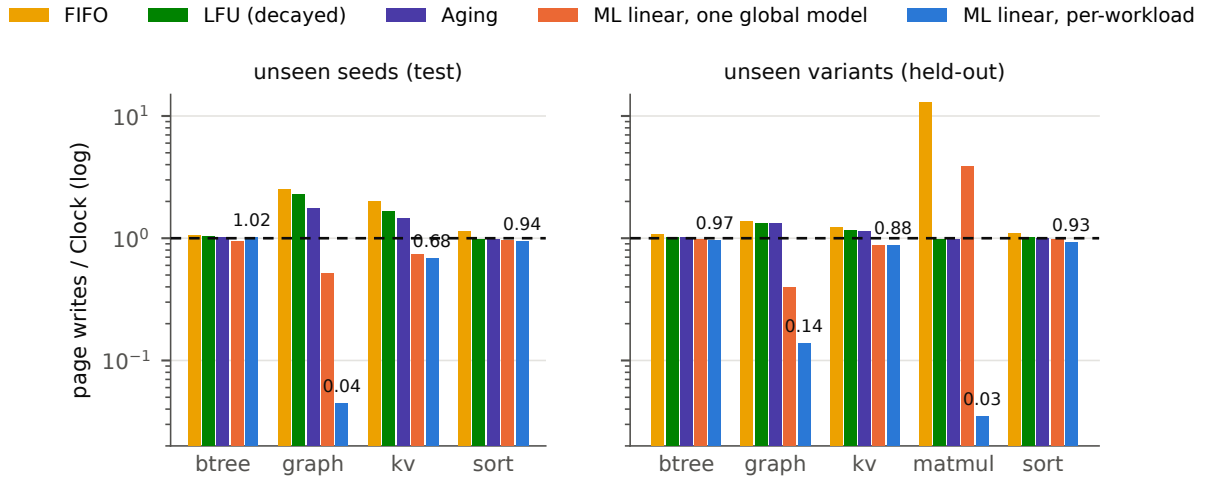


Figure 8: Measured in xv6: disk page writes relative to Clock (log scale).

sort 4.0% / 6.7% and matmul 2.7% (held-out). Disk writes drop by 95.5% / 86.2% on graph and 96.5% on matmul (Figure 8). The global model beats Clock everywhere except matmul (11.5% more faults).

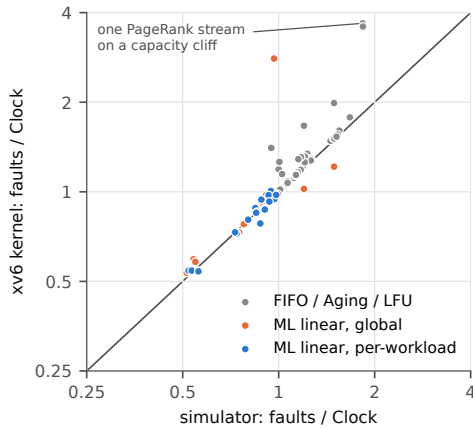


Figure 9: Each dot is one trace and policy: the simulator’s prediction (x) against the kernel’s measurement (y), both relative to Clock. Dots on the diagonal mean the simulator predicted the kernel exactly. The outlier is one PageRank trace that sits at a memory size where two more frames halve the faults; the kernel effectively has about two more frames for it than the simulator.

The simulator predicted the kernel well (Figure 9): for the ML models, Clock, Aging and LFU the median ratio of kernel to simulated faults is between 0.944 and 1.066 on every workload (the global model does better in the kernel, 0.834). So the simulation results of Sections 4.1–4.4 carry over to the real kernel. FIFO does worse in the kernel than simulated, most likely because there it also evicts the program’s own code and stack pages, which the traces do not include.

Workload	Split	pages in memory	timer ticks per eviction				ML time per eviction
			Clock	Aging	ML global	ML per-wl.	
btree	test	249	31.7	411.7	896.6	1011.6	101.2 μ s
btree	held-out	222	30.2	410.3	872.3	978.9	97.9 μ s
graph	test	160	55.6	277.0	543.4	594.4	59.4 μ s
graph	held-out	170	37.8	288.7	616.6	639.1	63.9 μ s
kv	test	62	28.0	130.0	250.8	275.2	27.5 μ s
kv	held-out	56	29.7	124.5	245.4	265.1	26.5 μ s
matmul	held-out	9	38.1	45.9	83.3	92.5	9.2 μ s
sort	test	10	35.1	48.4	82.2	88.1	8.8 μ s
sort	held-out	16	31.9	60.2	108.8	120.5	12.1 μ s

Table 10: Time to choose a victim in xv6, measured by the kernel’s 10 MHz timer (1 tick = 0.1 μ s of emulated time), mean per eviction at 10% memory. “Pages in memory” is how many pages the policy had to choose from. Last column: the per-workload ML model in microseconds.

The cost. The learned policy scores every page in memory at every eviction, so its decision time grows with the number of pages: about 43 ticks plus 3.9 per page (Table 10), from 8.8 μ s on sort (10 pages) to 101.2 μ s on btree (249 pages). That is 2.4–32.4 times Clock and 1.8–2.5 times Aging, and the main thing to improve.

Appendix: data and tables

Workload	Variant	Traces	Accesses (mean)	Pages	Split
btree	insert, lookup, mixed, mixed+WAL	5 each	210K–328K	1.8K–4.1K	train/val/test
	scan	5	413K	2.2K	held-out
graph	BFS+PageRank 1000 $\times$ 3	3	6.2M	999	train/val/test
	BFS+PageRank 2000 $\times$ 1	5	6.3M	2.0K	train/val/test
	PageRank 2000 $\times$ 2	3	6.1M	1.7K	train/val/test
	BFS 2000	3	3.2M	1.7K	held-out
kv	A, A+rehash, A+value sizes, B, C, D	5 each	100K–479K	527–749	train/val/test
	F	5	321K	528	held-out
matmul	naive / blocked, n = 64, 96	1 each	528K–2.0M	12–27	train
	naive / blocked, n = 128	1 each	4.2M–4.7M	48	held-out
sort	n = 20,000, 40,000	3 each	867K–1.9M	40–79	train/val/test
	n = 60,000	3	2.8M	118	held-out

Table A1: The 89 traces. “Pages” is the number of distinct pages a trace touches; memory sizes are 5/10/20% of it.

Model	Features (probation in scans)
btree	aging + idle + age + dirty + refaults (0)
graph	sfreq + idle + dirty + rdist (4)
kv	aging + idle + dirty + refaults + rdist (2)
matmul	aging + idle + dirty + refaults + rdist (0)
sort	ref + dirty + refaults + rdist (4)
global	idle + dirty + refaults + rdist (4)

Table A2: The selected kernel+refault models, chosen on validation. These are the weights the kernel runs.

Workload	Split	FIFO	Aging	LFU	ML global	ML per-wl.	ML K only
btree	test	1.143	1.116	1.127	0.843	0.823	0.824
graph	test	2.096	1.585	2.027	0.989	0.614	0.630
kv	test	1.284	1.107	1.175	0.875	0.871	0.908
sort	test	1.331	0.988	0.991	0.980	0.960	0.988
btree	held-out	1.087	1.063	1.073	0.893	0.854	0.893
graph	held-out	1.289	1.264	1.274	0.535	0.541	0.550
kv	held-out	1.312	1.133	1.187	0.878	0.875	0.924
matmul	held-out	1.820	0.999	0.999	1.115	0.973	0.999
sort	held-out	1.190	0.998	1.006	0.977	0.933	0.998

Table A3: All in-kernel results: faults relative to Clock at 10% memory (kernel counters, geometric mean over traces).

What	Where
Benchmarks	<code>user/btreebench.c</code> , <code>kvbench.c</code> , <code>graphbench.c</code> , <code>sortbench.c</code> , <code>matmulbench.c</code>
Trace collection	<code>tools/make_stream_manifest.py</code> , <code>run_stream_lanes.sh</code> , <code>check_streams.py</code> , <code>stream_dataset.py</code>
Simulator, training, selection	<code>tools/ml2/</code> (<code>pagesim.c</code> , <code>build_datasets.py</code> , <code>sweep_linear.py</code> , <code>final.py</code>)
Kernel policy	<code>kernel/vmpage.c</code> (<code>choose_ml</code>), <code>kernel/mlfeat.h</code> , <code>kernel/mlweights.h</code> , <code>user/vmrun.c</code>
Kernel evaluation	<code>tools/ml2/kernel_eval.py</code> , <code>kernel_vs_sim.py</code>
All results and models	<code>report/results2/</code> (every generated table: <code>tables.md</code>), <code>report/models2/</code>

Table A4: Where everything is.